

SNN-PDE

Key ideas

SNN-PDE structure

Methodology

SNN-PDE

d & d^*

**Hodge
Laplacian**

Mathematical Definitions

Manifold

Differential Forms

Hodge $*$

Exterior Derivative and its adjoint

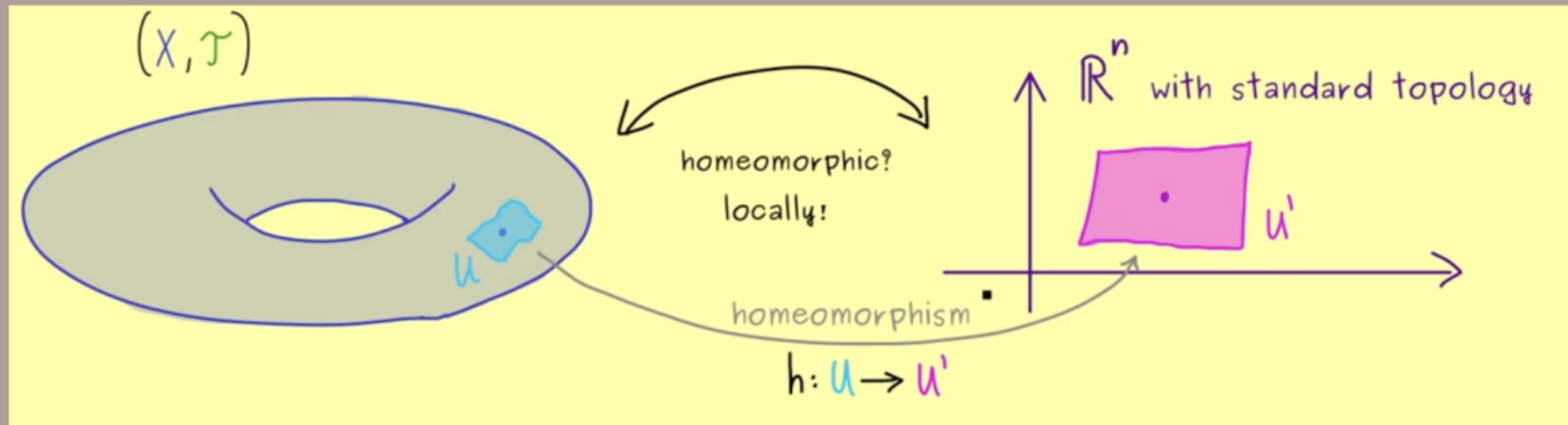
Manifold

**Differential
forms**

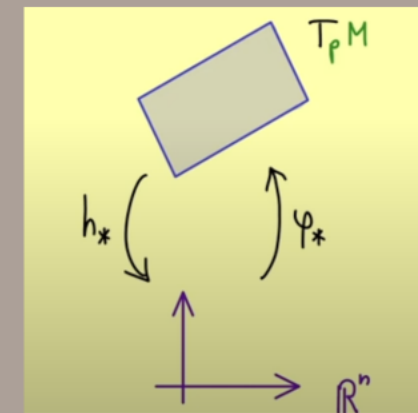
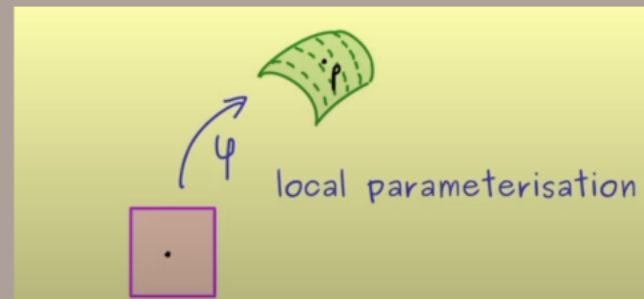
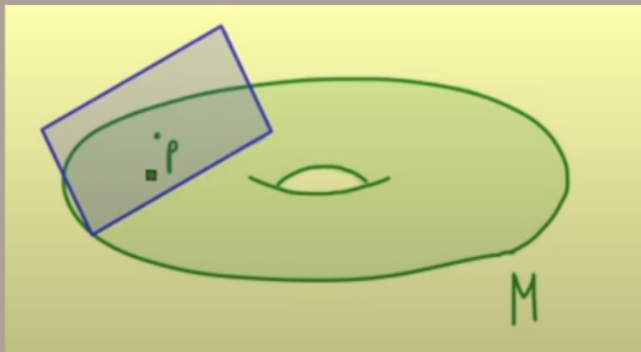
Hodge $*$

Manifold

Essential property - Locally Euclidean space



Tangent space at any point $p \in M$ can also be defined



d & d^*

**Hodge
Laplacian**

Mathematical Definitions

Manifold

Differential Forms

Hodge $*$

Exterior Derivative and its adjoint

Manifold

**Differential
forms**

Hodge $*$

Differential forms

0-form is a scalar function, e.g. $f(x) = ax^2 + bx + c$

1-form is a linear function $\omega: T_p \mathbb{R}^n \rightarrow \mathbb{R}$, e.g. 1-form in $\mathbb{R}^n$, $a_1 dx_1 + \dots + a_n dx_n$

Wedge product is a multiplication defined for k-forms.

Wedge product of two 1-forms are a 2-form, $\omega_1 \wedge \omega_2 : T_p \mathbb{R}^n \times T_p \mathbb{R}^n \rightarrow \mathbb{R}$

m-form is a function, $\omega: (T_p \mathbb{R}^n)^m \rightarrow \mathbb{R}$

Differential m-form on $\mathbb{R}^n$

$$\omega = \sum_I f_I dx_I, I = (i_1, \dots, i_m)$$

1-form acts on vector in $\mathbb{R}^2$:

$$dx(\langle a_1, a_2 \rangle) = a_1, \quad dy(\langle a_1, a_2 \rangle) = a_2$$

Example:

Suppose $\vec{v} = \langle a_1, a_2 \rangle, \vec{w} = \langle b_1, b_2 \rangle$

$dx \wedge dy(\vec{v}, \vec{w}) = \det \begin{pmatrix} dx(v) & dy(v) \\ dx(w) & dy(w) \end{pmatrix} = \det \begin{pmatrix} \vec{v} \\ \vec{w} \end{pmatrix}$, signed area of the parallelogram defined by $\vec{v}$ and $\vec{w}$

m-form acts on m-vectors in $\mathbb{R}^m$

$$(\omega_1 \wedge \cdots \wedge \omega_m)(v_1, \dots, v_m) = \begin{vmatrix} \omega_1(v_1) & \cdots & \omega_m(v_1) \\ \vdots & & \vdots \\ \omega_1(v_m) & \cdots & \omega_m(v_m) \end{vmatrix}$$

Example:

Let $\omega_1 = dx + 2dy, \omega_2 = dx - dz, \omega_3 = dx + dy + dz$

$v_1 = \langle 2, 1, 0 \rangle, v_2 = \langle -1, 3, -2 \rangle, v_3 = \langle 1, 0, 1 \rangle$

$$\omega_1 \wedge \omega_2 \wedge \omega_3(v_1, v_2, v_3) = \begin{vmatrix} 4 & 2 & 3 \\ 5 & 1 & 0 \\ 1 & 0 & 2 \end{vmatrix} = 15$$

Every m-form on $T_p \mathbb{R}^n$ can be written as $w = \sum_{1 \leq i_1 < \cdots < i_m \leq n} a_{i_1} \cdots a_{i_m} dx_{i_1} \wedge dx_{i_2} \wedge \cdots \wedge dx_{i_m}$

d & d^*

**Hodge
Laplacian**

Mathematical Definitions

Manifold

Differential Forms

Hodge $*$

Exterior Derivative and its adjoint

Manifold

**Differential
forms**

Hodge $*$

Hodge* Operator

$*$: $\Lambda^m(\mathbb{R}^n) \rightarrow \Lambda^{n-m}(\mathbb{R}^n)$, maps m-form to n-m form in $\mathbb{R}^n$

E.g. In $\mathbb{R}^2$, 0-form is mapped to 2-form, 2-form is mapped back to 0-form. 1-form remains 1-form.

Hodge Star Operator:

In $\mathbb{R}^n$,

$$* dx_I = dx_J, \text{ such that } dx_I \wedge dx_J = dx_1 \wedge \cdots \wedge dx_n$$

$$* \omega = \sum_I f_I (* dx_I)$$

Example:

$$*: \Lambda^2(\mathbb{R}^5) \rightarrow \Lambda^3(\mathbb{R}^5)$$

$$dx_I \wedge (* dx_I) = dx_1 \wedge \cdots \wedge dx_5$$

$$\omega = dx_1 \wedge dx_2 + 2dx_3 \wedge dx_4 + 7dx_1 \wedge dx_5$$

$$* \omega = dx_3 \wedge dx_4 \wedge dx_5 + 2dx_1 \wedge dx_2 \wedge dx_5 - 7dx_2 \wedge dx_3 \wedge dx_4$$

d & d^*

**Hodge
Laplacian**

Mathematical Definitions

Manifold

Differential Forms

Hodge $*$

Exterior Derivative and its adjoint

Manifold

**Differential
forms**

Hodge $*$

d, d^*

d maps differential m -form on $\mathbb{R}^n$ to differential $m+1$ -form on $\mathbb{R}^n$

d^* maps differential $m+1$ -form on $\mathbb{R}^n$ to differential m -form on $\mathbb{R}^n$

E.g. given a zero form $f(x_1, \dots, x_n)$, df is a 1-form

For differential m-forms:

Set $\omega = f dx_I$,

$$d\omega = \sum_{j=1}^n \frac{\partial f}{\partial x_j} dx_j \wedge dx_I$$

$$d(f dx_I) = \frac{\partial f}{\partial x_1} dx_1 \wedge dx_I + \cdots + \frac{\partial f}{\partial x_n} dx_n \wedge dx_I$$

Example:

$$\omega = x^2 y^2 dx \wedge dz + 2x^3 y z dy \wedge dz$$

$$d\omega = 2xy^2 dx \wedge dx \wedge dz + 2y dy \wedge dx \wedge dz + 6x^2 y z dx \wedge dy \wedge dz$$

$$=(6x^2 y z - 2y) dx \wedge dy \wedge dz$$

d & d^*

**Hodge
Laplacian**

Mathematical Definitions

Manifold

Differential Forms

Hodge $*$

Exterior Derivative and its adjoint

Manifold

**Differential
forms**

Hodge $*$

Hodge Laplacian Operator

maps differential m-form to m-form

$$\Delta_H = dd * \alpha + d * d\alpha$$

d & d^*

**Hodge
Laplacian**

Mathematical Definitions

Manifold

Differential Forms

Hodge $*$

Exterior Derivative and its adjoint

Manifold

**Differential
forms**

Hodge $*$

SNN-PDE

Key ideas

SNN-PDE structure

Methodology

SNN-PDE

**PDEs on
manifold**

Methodology

consider a smooth $(d-1)$ -dimensional manifold in $\mathbb{R}^d$

there's point $p \in \Gamma$ with coordinates $(p_1, \dots, p_{d-1})$

metric $g_p(u, v) = \sum u_i v_i$

$d=3$ in this case

**Gradient
Operator**

**Hodge
Laplacian**

Gradient Operator

Consider a continuous differentiable function $f: \Gamma \rightarrow \mathbb{R}$

The corresponding gradient operator is given by $\nabla_{\Gamma} f = \sum_i a^i \frac{\partial f}{\partial p_i}$, where $a^i = \sum_j g^{ij} \frac{\partial f}{\partial p_j}$, e.g. $a^1 = \sum_j g^{1j} \frac{\partial f}{\partial p_j} = g^{11} \frac{\partial f}{\partial p_1} + g^{12} \frac{\partial f}{\partial p_2}$

g^{ij} are the elements of inverse of metric tensor $G = \{g_{ij}\}_{i,j=1}^2$

$$g_{ij} = g \left(\frac{\partial}{\partial p_1} + \frac{\partial}{\partial p_2} \right) \quad 1 \leq i, j \leq 2$$

**PDEs on
manifold**

Methodology

consider a smooth $(d-1)$ -dimensional manifold in $\mathbb{R}^d$

there's point $p \in \Gamma$ with coordinates $(p_1, \dots, p_{d-1})$

metric $g_p(u, v) = \sum u_i v_i$

$d=3$ in this case

**Gradient
Operator**

**Hodge
Laplacian**

Hodge Laplacian Operator

$$\Delta_H f = -\sum_{ij} \frac{1}{|G|} \frac{\partial}{\partial p_i} \left(|G| g^{ij} \frac{\partial f}{\partial p_j} \right).$$

scalar function f is a 0-form, Hodge Laplacian is given by $\Delta_H f = dd^*f + d^*df$

since $d^*f=0$ then $\Delta_H f$ reduces to d^*df , $d^*df = * d * df$ by definition

which is equivalent to $\text{divgrad } f$, surface laplacian times -1

**PDEs on
manifold**

Methodology

consider a smooth $(d-1)$ -dimensional manifold in $\mathbb{R}^d$

there's point $p \in \Gamma$ with coordinates $(p_1, \dots, p_{d-1})$

metric $g_p(u, v) = \sum u_i v_i$

$d=3$ in this case

**Gradient
Operator**

**Hodge
Laplacian**

PDEs on Manifold

PDEs are defined as such, $\frac{\partial u}{\partial t}(s, t) = F(s, u, \nabla_{\Gamma s} u, \Delta_{Hs} u), s \in \Gamma$

Discretise the manifold into $\mathcal{N}$ locations $\{s_1, \dots, s_{\mathcal{N}}\}$

The function F is approximated by $\hat{F}$ at each discrete point and its neighboring points

$\hat{F}$ is represented by $\hat{F}_{\theta}$, where f_M computes messages based on the states of the node and its neighbors and f_U aggregates the messages and updates the node's state.

**PDEs on
manifold**

Methodology

consider a smooth $(d-1)$ -dimensional manifold in $\mathbb{R}^d$

there's point $p \in \Gamma$ with coordinates $(p_1, \dots, p_{d-1})$

metric $g_p(u, v) = \sum u_i v_i$

$d=3$ in this case

**Gradient
Operator**

**Hodge
Laplacian**

SNN-PDE

Key ideas

SNN-PDE structure

Methodology

SNN-PDE



Basic Structure

Dual Graph Convolutional
Encoder

+

Multi-Channel Simplicial
Convolutional Encoder

Dual Graph
Convolutional
encoder

Multi-Channel
Simplicial
Convolutional
Encoder

Dual Graph Convolutional encoder

graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{S})$

$\mathcal{V}$: nodes, $\mathcal{E}$: edges , $\mathbf{S}$: attribute matrix

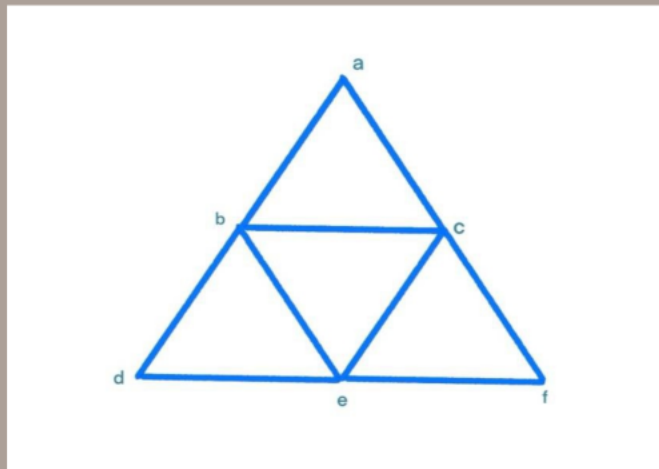
Each Node v_i contains a d_c -dimensional vector of attributes

$$\mathbf{S} = \begin{pmatrix} \cdots & node1 & \cdots \\ \cdots & node2 & \cdots \\ \cdots & \vdots & \cdots \\ \cdots & node\mathcal{N} & \cdots \end{pmatrix} \in \mathbb{R}^{\mathcal{N}} \times d_c$$

Connectivity of graph can be encoded in a form of an adjacency matrix, $\mathbf{A} \in \mathbb{R}^{\mathcal{N} \times \mathcal{N}}$

Multi-Hop Grid-Structural Graph Convolution

Computational Illustration



Finally, the multi-hop grid structural graph convolution layer:

$$\mathbf{Z}_{\mathcal{G}}^{(\ell+1)} = \sigma \left(\tilde{\mathbf{L}}_{\mathcal{G}}^{[r]} \mathbf{Z}_{\mathcal{G}}^{(\ell)} \Theta_{\mathcal{G}} \right)$$

Adjacency Matrix, $\mathbf{A} =$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
<i>a</i>	0	1	1	0	0	0
<i>b</i>	1	0	1	1	1	0
<i>c</i>	1	1	0	0	1	1
<i>d</i>	0	1	0	0	1	0
<i>e</i>	0	1	1	1	0	1
<i>f</i>	0	0	1	0	1	0

Adjacency matrix with self-loop, $\tilde{\mathbf{A}} =$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
<i>a</i>	1	1	1	0	0	0
<i>b</i>	1	1	1	1	1	0
<i>c</i>	1	1	1	0	1	1
<i>d</i>	0	1	0	1	1	0
<i>e</i>	0	1	1	1	1	1
<i>f</i>	0	0	1	0	1	1

$\tilde{\mathbf{D}} =$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
<i>a</i>	3	0	0	0	0	0
<i>b</i>	0	5	0	0	0	0
<i>c</i>	0	0	5	0	0	0
<i>d</i>	0	0	0	3	0	0
<i>e</i>	0	0	0	0	5	0
<i>f</i>	0	0	0	0	0	3

Graph Laplacian, $\mathbf{L}_{\mathcal{G}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$

Weighted multi-hop grid structural graph Laplacian, $\tilde{\mathbf{L}}_{\mathcal{G}}^{[r]} = \sum_{\tau=1}^r \alpha_{\tau} \times \mathbf{L}_{\mathcal{G}}^{\tau}$

Multi-Hop Adaptive Graph Convolution

Graph Convolution on Adaptive Graph Strutcure:

$$\mathbf{Z}_{\mathcal{A}}^{(\ell+1)} = \sigma \left(\widetilde{\mathbf{\Xi}}_{\mathcal{A}}^{[r]} \mathbf{Z}_{\mathcal{A}}^{(\ell)} \mathbf{\Theta}_{\mathcal{A}} \right)$$

Node embedding dictionary, $E^\phi = \begin{pmatrix} \cdots & e_1^\phi & \cdots \\ \cdots & e_2^\phi & \cdots \\ \cdots & \vdots & \cdots \\ \cdots & e_N^\phi & \cdots \end{pmatrix} \in \mathbb{R}^{N \times d_e}$

A non-negative **normalized** function, $\Xi_{uv} = \mathcal{G}(e_u^\phi, e_v^\phi) = \frac{\exp(\text{ReLU}(e_u^\phi (e_v^\phi)^\top))}{\sum_{u=1}^N \exp(\text{ReLU}(e_u^\phi (e_v^\phi)^\top))}$, for each pair u, v .

Example:

Suppose there's three nodes u, v, w , with the following similarity scores after ReLU –

$$\text{ReLU} \left(e_u^\phi (e_v^\phi)^\top \right) = 2$$

$$\text{ReLU} \left(e_u^\phi (e_w^\phi)^\top \right) = 1$$

$$\text{ReLU} \left(e_u^\phi (e_u^\phi)^\top \right) = 3$$

The exponentiated scores are-

$$\exp(2)$$

$$\exp(1)$$

$$\exp(3)$$

The normalized similarity score $\Xi_{uv} = \frac{\exp(2)}{\exp(2)+\exp(1)+\exp(3)}$

Weighted multi-hop adaptive graph Laplacian $\widetilde{\mathbf{\Xi}}^{[r']} = \sum_{\tau=1}^{r'} \beta_\tau \times \mathbf{\Xi}^\tau$

* $\mathbf{\Xi}^\tau$ represents the similarity matrix at the τ -th hop



Basic Structure

Dual Graph Convolutional
Encoder

+

Multi-Channel Simplicial
Convolutional Encoder

Dual Graph
Convolutional
encoder

Multi-Channel
Simplicial
Convolutional
Encoder

Simplicial Complexes

0-simplex, a point

1-simplex, a line segment defined by its two end points

2-simplex, a triangle

3-simplex, a tetrahedron

k -simplex is the convex hull of $k+1$ affinely independent points (vertices)

Only have up to k -simplex in k -dimensional space

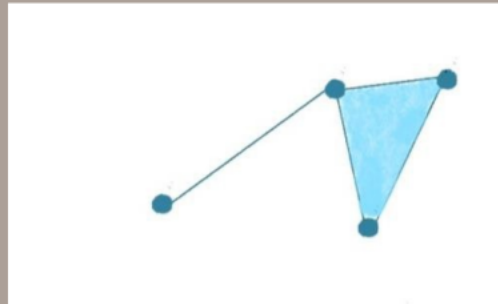
Simplicial complex is a collection of simplices where the intersection of any two simplices is a simplex and every face of every simplex in the complex is also in the complex

For simplicial 1-complexes, we write $G = (V, E)$

For simplicial 2-complexes, we write $K = (V, E, F)$

$V = \text{vertices}, E = \text{edges}, F = \text{faces}$

Example:



Discrete version of Hodge- Laplacian on Graphs

Orientation of k-simplex can be defined by selecting some order for its nodes.

Oriented k simplices form basis of C^k

Example

In C^1 ,

Vertices: v_0, v_1, v_2

Edges: $[v_0, v_1], [v_1, v_2], [v_0, v_2]$

Basis Elements: $\{[v_0, v_1], [v_1, v_2], [v_0, v_2]\}$, oriented edges

A vector space of real numbers C^k is constructed to study the diffusion among higher-order substructures of a graph $\mathcal{G}$. Elements of C^k are called k-chains.

Co-chains are dual of k-chain, $\omega: C^k \rightarrow \mathbb{R}$ (natural counterpart of differential forms)

Boundary Operator, $\partial_k: C^k \rightarrow C^{k-1}$ (discrete analogue of codifferential operator d^*)

Matrix representation B_k

Co-Boundary Operator $\partial_k^T: C^k \rightarrow C^{k+1}$ (discrete analogue of exterior derivative d)

Matrix representation B_k^T

Operator over oriented k-simplices $L_k: C^k \rightarrow C^k$, **Discrete Hodge- Laplacian**

$$L_k = B_k^T B_k + B_{k+1} B_{k+1}^T$$

Multi-Channel Simplicial Convolutional Encoder

- ✧ Extract Node embeddings from k-simplex

Using simplicial convolution Operation,

$$\mathbf{Z}_{k,\mathcal{S}}^{(\ell+1)} = (\mathbf{B}_1^\top \cdots \mathbf{B}_k^\top) \left(\sigma \left(\mathbf{L}_k \mathbf{Z}_{k,\mathcal{S}}^{(\ell)} \boldsymbol{\Theta}_{k,\mathcal{S}}^{(\ell)} \right) \right)$$

$\mathbf{L}_k$: k-Hodge Laplacian

$\mathbf{B}_k$: Incidence matrix of Boundary Operator

$\mathbf{Z}_{k,\mathcal{S}}^{(0)}$: initial input activation

$\mathbf{Z}_{1,\mathcal{S}}^{(0)}$, $\mathbf{Z}_{2,\mathcal{S}}^{(0)}$ represents the edge feature matrix and triangle feature matrix respectively.

Obtain $\mathcal{K}$ embeddings, $\{\mathbf{Z}_{1,\mathcal{S}}^{(\ell+1)}, \dots, \mathbf{Z}_{k,\mathcal{S}}^{(\ell+1)}, \dots, \mathbf{Z}_{\mathcal{K},\mathcal{S}}^{(\ell+1)}\}$

- ✧ Compute attention coefficients

$$\begin{aligned} \rho_k &= \text{softmax}_i \left(\gamma_{\text{Att}} \tanh \left(\boldsymbol{\Phi} \mathbf{Z}_{k,\mathcal{S}} \right) \right) \\ &= \frac{\exp \left(\gamma_{\text{Att}} \tanh \left(\boldsymbol{\Phi} \mathbf{Z}_{k,\mathcal{S}} \right) \right)}{\sum_{k \in \{1, \dots, \mathcal{K}\}} \exp \left(\gamma_{\text{Att}} \tanh \left(\boldsymbol{\Phi} \mathbf{Z}_{k,\mathcal{S}} \right) \right)} \end{aligned}$$

- ✧ Combine all embeddings to obtain final Embedding $\mathbf{Z}_{\mathcal{S}}$

$$\mathbf{Z}_{\mathcal{S}} = \rho_1 \times \mathbf{Z}_{1,\mathcal{S}} + \cdots + \rho_k \times \mathbf{Z}_{k,\mathcal{S}} + \cdots + \rho_{\mathcal{K}} \times \mathbf{Z}_{\mathcal{K},\mathcal{S}}.$$



Basic Structure

Dual Graph Convolutional
Encoder

+

Multi-Channel Simplicial
Convolutional Encoder

Dual Graph
Convolutional
encoder

Multi-Channel
Simplicial
Convolutional
Encoder

SNN-PDE

Key ideas

SNN-PDE structure

Methodology

SNN-PDE

■ SNN-PDE

Combine the embeddings from both the graph convolutional encoder and multi-channel simplicial convolutional encoder,

$$\mathbf{Z}_{\hat{F}_\theta} = \gamma_G \times \mathbf{Z}_G + \gamma_{\mathcal{A}} \times \mathbf{Z}_{\mathcal{A}} + \gamma_S \times \mathbf{Z}_S$$

Use the final embeddings as input to the PDE surrogate $\hat{F}_\theta$

The system states can be estimated,

$$\hat{\mathbf{u}}(t) = (\hat{u}(s_1, t), \hat{u}(s_2, t), \dots, \hat{u}(s_N, t))$$

SNN-PDE

Key ideas

SNN-PDE structure

Methodology

SNN-PDE